

1 Objectivos - Uso e implementação de estruturas de dados do programa da disciplina

Uso e implementação de estruturas de dados do programa da disciplina (Listas, Pilhas, Tabelas de Hash, etc), para desenvolvimento de uma aplicação modular em linguagem C, reutilização e adaptação dos TADs (Tipos Abstratos de Dados) desenvolvidos em contexto de sala de aula.

2 O trabalho

2.1 Descrição

O objetivo é implementar um editor de texto baseado em linhas (**ED-LIN**). O editor utiliza um **Cursor** para navegação entre linhas e permite a manipulação dinâmica do texto com suporte a reversão das operações (*Undo*) e verificação ortográfica (*Spellchecker*). Este trabalho está dividido em 3 partes. A parte 1, corresponde ao editor, a parte 2 à correcção ortográfica e a parte 3 à reversão das operações. É opcional a realização das partes 2 e 3. A cotação da parte 1 é 15 valores, da parte 2, 5 valores e da parte 3, 5 valores.

```
----- Editor -----  
1 : É urgente destruir certas palavras,  
2 : ódio, solidão e crueldade,  
3 : alguns lamentos,  
4 > muitas espadas.  
-----
```

Parte 1: Editor de Texto e Cursor

A aplicação deve gerir um documento como uma sequência de linhas. O **Cursor** indica a posição ativa para edição.

- **Inserção:**

- `InsertInCursor "texto"`: Insere na posição atual do cursor.
- `InsertInLine <n> "texto"`: Insere na linha indicada pelo índice n .
- `InsertEnd "texto"`: Adiciona ao final do documento.

- **Remoção:**

- `DeleteLine`: Remove a linha do cursor.
- `DeleteCursor <n>`: Remove a linha no índice especificado.

- **Edição:**

- `EditCursor "texto"` edita a linha do cursor, substituindo-a pelo texto dado
- `EditLine <n> "texto"` substitui o conteúdo da n -ésima linha.

- **Navegação:**

- `GoUp`, desloca o cursor para a linha acima
- `GoDown` desloca o cursor para a linha abaixo.

- **Visualização:**

- `Print` apresenta o documento numerado; a linha do cursor deve ser assinalada (ex: `>`).
- `PrintCursor`, mostra a linha do cursor

- **Pesquisa:**

- `Search "texto"` imprime o número e o conteúdo da linha que contém o padrão; se não existir, indica "not found".

Nota: Line Wrap: Nenhuma linha pode exceder **80 caracteres**. Se ultrapassar, o excedente deve passar automaticamente para uma nova linha inserida imediatamente abaixo.

Parte 2: Verificação Ortográfica (Spellchecker)

Integração de um dicionário para validar o texto.

- Carregar o ficheiro `portuguese.txt` para uma estrutura de pesquisa rápida.
- **Comando `spell`**: Lista as palavras do documento que não constam no dicionário, removendo pontuação e ignorando a capitalização (*case-insensitive*). A listagem de palavras deve respeitar a regra: uma única palavra por linha, sendo indicado o número da linha e a palavra incorreta.

Parte 3: Sistema de Undo

- Todas as operações de alteração (`Insert`, `Delete` e `edit`) devem ser passíveis de reversão.
- O comando `undo` restaura o estado anterior do documento e a posição do cursor.

Estruturas de Dados Recomendadas

Finalidade	Estrutura sugerida
Editor	Lista Duplamente ligada
Spellcheck	Tabela de Hash
Undo	Stack

2.2 Implementação

Cada estrutura de dados utilizada deve ser devidamente implementada. Pode adaptar as implementações fornecidas nas aulas práticas (ajustando o tipo `ElementType`). A implementação das estruturas base deve preceder a do editor, Undo ou spellcheck. Funcionalidades adicionais serão bonificadas se forem coerentes, úteis e bem integradas. O trabalho é realizado em grupos de 2 elementos(± 1). A segunda e terceira partes do trabalho, podem ser realizadas somente por alguns elementos do grupo.

2.3 Entrega

- Submissão: 1ª parte - até 9 de junho de 2026, via Moodle.
- Submissão: 2ª parte/3ª parte - até 2 de Julho de 2026
- Entrega: ficheiro `.zip` com:
 - Todos os ficheiros do projeto.
 - Relatório em pdf.
 - Nome do ficheiro: NNNNN-MMMMM-Editor1.zip (Editor2,Editor2.3) (usando os números dos alunos).
 - Submissão feita pelo aluno com o menor número.